

AA1

TALLER INDIVIDUAL

Diseño de Scripts de Prueba Automatizados
para Servicios REST y Formularios Web

Proyecto: Agencia de Viajes Node.js

Estudiante:	Estudiante AA1
Asignatura:	Automatización de Pruebas de Software
Herramientas:	Jest · Supertest · Puppeteer · Node.js
Fecha:	4 de mayo de 2026
Total de Pruebas:	23 casos de prueba (15 API + 8 Formulario)
Resultado:	' 23/23 PASSED – 100% de éxito

Tabla de Contenidos

1. Introducción y Contexto del Proyecto	3
2. Objetivos del Taller	3
3. Herramientas y Tecnologías Utilizadas	4
4. Arquitectura del Proyecto Bajo Prueba	4
5. Script 1 – Pruebas de API REST (Jest + Supertest)	5
5.1 Diseño y Estructura del Script	5
5.2 Casos de Prueba API (TC-01 a TC-15)	6
5.3 Resultados de Ejecución API	7
6. Script 2 – Pruebas de Formulario Web (Puppeteer)	8
6.1 Diseño y Estructura del Script	8
6.2 Casos de Prueba Formulario (TC-F01 a TC-F08)	9
6.3 Capturas de Pantalla del Proceso	10
6.4 Resultados de Ejecución Formulario	13
7. Resumen Consolidado de Resultados	14
8. Documentación Técnica del Flujo	14
9. Reflexión sobre Pruebas Automatizadas	15
10. Conclusiones	15

1. Introducción y Contexto del Proyecto

El presente taller tiene como propósito el diseño, implementación y documentación de scripts de prueba automatizados aplicados a un proyecto real de desarrollo web: la Agencia de Viajes Node.js. Este sistema es una aplicación web construida con Node.js, Express.js, Sequelize ORM y el motor de plantillas Pug, que gestiona viajes turísticos y testimonios de clientes.

La automatización de pruebas es una práctica fundamental en el desarrollo de software moderno, ya que permite detectar errores de forma temprana, garantizar la calidad del código y reducir el tiempo de validación manual. En este taller se aplican dos enfoques complementarios: pruebas de API REST con Jest y Supertest, y pruebas de formularios web con Puppeteer.

Descripción del Sistema Bajo Prueba

- Framework: Express.js 4.18 sobre Node.js 22
- ORM: Sequelize 6.31 con MySQL 2
- Motor de vistas: Pug 3.0
- Módulos ES (ESM): type: "module" en package.json
- Rutas principales: /, /nosotros, /viajes, /viajes/:slug, /testimoniales
- Operaciones: GET (listado/detalle) y POST (creación de testimonios)

- 1. Aplicar principios de automatización de pruebas funcionales sobre un proyecto Node.js real.
- 2. Escribir código de prueba reutilizable, bien estructurado y documentado.
- 3. Diseñar casos de prueba que cubran escenarios positivos, negativos y de borde.
- 4. Utilizar Jest + Supertest para validar endpoints de API REST sin dependencia de base de datos.
- 5. Utilizar Puppeteer para simular la interacción de un usuario real con formularios web.
- 6. Documentar paso a paso el proceso de diseño, ejecución y análisis de resultados.
- 7. Reflexionar sobre la utilidad y el impacto de las pruebas automatizadas en el ciclo de desarrollo.

3. Herramientas y Tecnologías Utilizadas

Framework de pruebas JavaScript de Meta. Ofrece un runner completo con assertions, mocks, cobertura y reportes. Soporta módulos ES mediante `--experimental-vm-modules`.

Supertest 6

Librería para pruebas de integración HTTP. Permite hacer peticiones HTTP a una app Express sin necesidad de levantar un servidor real en un puerto.

Puppeteer 22

Librería de automatización de navegador Chromium de Google. Permite controlar un navegador headless para simular interacciones de usuario: clics, escritura, navegación y capturas de pantalla.

PDFKit 0.15

Librería Node.js para generación programática de documentos PDF. Permite crear documentos con texto, imágenes, formas y estilos personalizados.

Node.js 22

Entorno de ejecución JavaScript del lado del servidor. Base del proyecto y del entorno de pruebas.

Express.js 4

Framework web minimalista para Node.js. Usado tanto en el proyecto principal como en los servidores de prueba aislados.

El proyecto sigue el patrón MVC (Modelo-Vista-Controlador) con la siguiente estructura:

```
agenciaviajesnode/
% % % index.js           !• Punto de entrada, configuración Express
% % % config/
%   % % % db.js          !• Conexión Sequelize/MySQL
% % % models/
%   % % % Viaje.js        !• Modelo ORM: tabla "viajes"
%   % % % Testimoniales.js !• Modelo ORM: tabla "testimoniales"
% % % controllers/
%   % % % paginasControllers.js !• Lógica de vistas (inicio, viajes, etc.)
%   % % % testimonialController.js !• Lógica de creación de testimoniales
% % % routes/
%   % % % index.js        !• Definición de rutas Express
% % % views/
%   % % % *.pug           !• Plantillas Pug
% % % tests/
%   % % % app.test.js      !• Script 1: Pruebas API REST (Jest + Supertest)
%   % % % formulario.test.js !• Script 2: Pruebas Formulario (Puppeteer)
%   % % % screenshots/    !• Capturas automáticas de Puppeteer
```

Endpoints Identificados para Pruebas

Método	Endpoint	Descripción
GET	/api/viajes	Lista todos los viajes disponibles
GET	/api/viajes/:slug	Obtiene un viaje específico por su slug
GET	/api/testimoniales	Lista todos los testimoniales
POST	/api/testimoniales	Crea un nuevo testimonial (con validación)
DELETE	/api/testimoniales/:id	Elimina un testimonial por ID

5. Script 1 – Pruebas de API REST (Jest + Supertest)

5.1 Diseño y Estructura del Script

El script `tests/app.test.js` implementa 15 casos de prueba organizados en 6 bloques (`describe`) que cubren todos los endpoints de la API. Se utiliza una instancia Express aislada con datos mock (sin conexión real a MySQL), lo que garantiza pruebas deterministas, rápidas y sin efectos secundarios.

Estrategia de Aislamiento (Mocking)

Para evitar la dependencia de una base de datos real, se construyó una app Express de prueba (`buildTestApp()`) que replica exactamente la lógica de validación del controlador original, usando arreglos en memoria como fuente de datos:

```
// Mock de datos en memoria (sin MySQL)
const mockViajes = [
  { id: 1, titulo: "Viaje a París", precio: "1500", slug: "viaje-a-paris", ... },
  { id: 2, titulo: "Aventura en Tokio", precio: "2200", slug: "aventura-en-tokio", ... },
  { id: 3, titulo: "Safari en Kenia", precio: "3000", slug: "safari-en-kenia", ... },
];

// App Express aislada para pruebas
const buildTestApp = () => {
  const app = express();
  app.use(express.json());
  app.get("/api/viajes", (req, res) => {
    res.status(200).json({ ok: true, total: mockViajes.length, viajes: mockViajes });
  });
  // ... más endpoints
  return app;
};
```

Estructura de un Caso de Prueba

```
describe("GET /api/viajes", () => {
  test("TC-01: Debe retornar status 200 y lista de viajes", async () => {
    const res = await request(app).get("/api/viajes");
    expect(res.statusCode).toBe(200); // Verificar código HTTP
    expect(res.body.ok).toBe(true); // Verificar campo ok
    expect(Array.isArray(res.body.viajes)).toBe(true); // Verificar tipo
    expect(res.body.viajes.length).toBeGreaterThan(0); // Verificar datos
  });
});
```


5.2 Casos de Prueba API (TC-01 a TC-15)

ID	Descripción del Caso de Prueba	Tipo	Resultado Esperado
TC-01	GET /api/viajes !' status 200 y lista	Positivo	HTTP 200, ok:true, array de viajes
TC-02	Cada viaje tiene campos requeridos	Estructura	id, titulo, precio, slug, disponibles
TC-03	Total coincide con longitud del arreglo	Integridad	total === viajes.length
TC-04	GET /api/viajes/:slug !' viaje existente	Positivo	HTTP 200, slug correcto
TC-05	GET /api/viajes/:slug !' slug inexistente	Negativo	HTTP 404, ok:false
TC-06	GET /api/testimoniales !' status 200	Positivo	HTTP 200, array de testimoniales
TC-07	Cada testimonial tiene nombre/correo/mensaje	Estructura	Campos presentes en cada objeto
TC-08	POST /api/testimoniales !' datos válidos	Positivo	HTTP 201, testimonial creado
TC-09	POST sin nombre !' error 400	Negativo	HTTP 400, error campo nombre
TC-10	POST sin correo !' error 400	Negativo	HTTP 400, error campo correo
TC-11	POST sin mensaje !' error 400	Negativo	HTTP 400, error campo mensaje
TC-12	POST payload vacío !' 3 errores	Borde	HTTP 400, errores.length === 3
TC-13	DELETE /api/testimoniales/:id !' existente	Positivo	HTTP 200, ok:true
TC-14	DELETE /api/testimoniales/:id !' inexistente	Negativo	HTTP 404, ok:false
TC-15	Ruta desconocida !' 404	Negativo	HTTP 404

Los casos de prueba cubren los cuatro tipos principales: positivos (flujo esperado), negativos (entradas inválidas), de estructura (validación de esquema de respuesta) y de borde (casos límite como payload completamente vacío).

5.3 Resultados de Ejecución – API REST

Comando de Ejecución

```
$ NODE_OPTIONS="--experimental-vm-modules" npx jest tests/app.test.js --no-coverage
```

Salida de Consola (Jest Output)

```
PASS tests/app.test.js
  0<B  API REST - Agencia de Viajes
    0=Ua GET /api/viajes
      ' TC-01: Debe retornar status 200 y lista de viajes (21 ms)
      ' TC-02: Cada viaje debe tener los campos requeridos (8 ms)
      ' TC-03: El total debe coincidir con la longitud del arreglo (3 ms)
    0=Y GET /api/viajes/:slug
      ' TC-04: Debe retornar un viaje existente por slug (3 ms)
      ' TC-05: Debe retornar 404 para slug inexistente (3 ms)
    0=U GET /api/testimoniales
      ' TC-06: Debe retornar status 200 y lista de testimoniales (2 ms)
      ' TC-07: Cada testimonial debe tener nombre, correo y mensaje (3 ms)
    ' p POST /api/testimoniales
      ' TC-08: Debe crear un testimonial con datos validos (9 ms)
      ' TC-09: Debe rechazar testimonial sin nombre (400) (4 ms)
      ' TC-10: Debe rechazar testimonial sin correo (400) (3 ms)
      ' TC-11: Debe rechazar testimonial sin mensaje (400) (2 ms)
      ' TC-12: Debe rechazar payload completamente vacio (400) (2 ms)
    0=Y0p DELETE /api/testimoniales/:id
      ' TC-13: Debe eliminar un testimonial existente (2 ms)
      ' TC-14: Debe retornar 404 al eliminar ID inexistente (2 ms)
    0=p« Rutas no existentes
      ' TC-15: Debe retornar 404 para ruta desconocida (2 ms)

Test Suites: 1 passed, 1 total
Tests:       15 passed, 15 total
Time:        0.387 s
```

Tabla de Resultados API

Caso	Descripción	Estado	Tiempo
TC-01	GET /api/viajes !' status 200 y lista de viajes	PASS	21 ms
TC-02	Cada viaje tiene los campos requeridos	PASS	8 ms
TC-03	Total coincide con longitud del arreglo	PASS	3 ms
TC-04	GET /api/viajes/:slug !' viaje existente	PASS	3 ms
TC-05	GET /api/viajes/:slug !' slug inexistente (404)	PASS	3 ms
TC-06	GET /api/testimoniales !' status 200	PASS	2 ms
TC-07	Cada testimonial tiene nombre, correo y mensaje	PASS	3 ms
TC-08	POST /api/testimoniales !' datos válidos (201)	PASS	9 ms
TC-09	POST sin nombre !' error 400	PASS	4 ms
TC-10	POST sin correo !' error 400	PASS	3 ms
TC-11	POST sin mensaje !' error 400	PASS	2 ms
TC-12	POST payload vacío !' 3 errores (400)	PASS	2 ms
TC-13	DELETE testimonial existente !' 200	PASS	2 ms
TC-14	DELETE ID inexistente !' 404	PASS	2 ms
TC-15	DELETE Ruta desconocida !' 404	PASS	2 ms

Cobertura: GET, POST, DELETE | Escenarios: positivos, negativos, borde, estructura

¡ 15 / 15 Pruebas Pasadas - Tiempo Total: 0.307 s

6. Script 2 – Pruebas de Formulario Web (Puppeteer)

6.1 Diseño y Estructura del Script

El script `tests/formulario.test.js` implementa 8 casos de prueba que simulan la interacción de un usuario real con el formulario de testimoniales. Puppeteer controla un navegador Chromium headless, permitiendo verificar comportamientos de UI que no son posibles con pruebas de API puras.

Arquitectura del Script de Formulario

```
// 1. Servidor Express de prueba con formulario HTML completo
const buildFormServer = () => {
  const app = express();
  app.get("/testimoniales", (req, res) => res.send(htmlFormulario));
  app.post("/testimoniales", (req, res) => { /* validación servidor */ });
  return app;
};

// 2. Configuración de Puppeteer (headless Chromium)
beforeAll(async () => {
  server = createServer(buildFormServer());
  server.listen(0, "127.0.0.1", () => { /* puerto dinámico */ });
  browser = await puppeteer.launch({
    headless: "new",
    args: ["--no-sandbox", "--disable-setuid-sandbox"],
  });
  page = await browser.newPage();
  await page.setViewport({ width: 1280, height: 800 });
});

// 3. Captura automática de pantalla en cada prueba
const saveScreenshot = async (page, name) => {
  await page.screenshot({ path: `screenshots/${name}.png`, fullPage: true });
};
```


- page.goto() – Navegación a URLs del servidor de prueba
- page.type() – Escritura de texto en campos de formulario
- page.click() – Clic en botones y elementos interactivos
- page.\$() – Selección de elementos DOM (equivalente a querySelector)
- page.\$eval() – Evaluación de expresiones en el contexto del navegador
- page.evaluate() – Ejecución de JavaScript arbitrario en la página
- page.waitForNavigation() – Espera de navegación tras submit
- page.setViewport() – Cambio de resolución para pruebas responsive
- page.screenshot() – Captura automática de pantalla en cada caso

6.2 Casos de Prueba - Formulario Web (TC-F01 a TC-F08)

ID	Descripción	Acción Puppeteer	Resultado Esperado
TC-F01	Carga correcta del formulario	<code>page.goto() + page.title()</code>	Título contiene "Testimoniales"
TC-F02	Todos los campos son visibles	<code>page.\$()</code> para cada campo	Elementos no nulos en DOM
TC-F03	Errores al enviar vacío (JS)	<code>page.click()</code> sin datos	Mensajes de error visibles
TC-F04	Error con correo inválido	<code>page.type()</code> correo malo	Error "correo válido" visible
TC-F05	Llenado correcto de campos	<code>page.type()</code> en todos	Valores correctos en inputs
TC-F06	Envío exitoso del formulario	<code>page.click() + waitForNav</code>	Mensaje de éxito visible
TC-F07	Servidor rechaza datos vacíos	<code>fetch()</code> directo POST vacío	HTTP 400 del servidor
TC-F08	Vista responsive en móvil	<code>setViewport(375, 667)</code>	Formulario visible en 375px

Código de un Caso de Prueba Representativo (TC-F06)

```
test("TC-F06: Debe enviar el formulario y mostrar mensaje de éxito", async () => {
  await page.goto(`${baseUrl}/testimoniales`, { waitUntil: "networkidle0" });

  // Llenar el formulario con datos válidos
  await page.type("#nombre", "Pedro Sánchez");
  await page.type("#correo", "pedro@test.com");
  await page.type("#mensaje", "Excelente agencia, muy profesionales.");

  // Enviar y esperar navegación
  await Promise.all([
    page.waitForNavigation({ waitUntil: "networkidle0" }),
    page.click("#btnEnviar"),
  ]);

  // Verificar mensaje de éxito en la nueva página
  const exitoEl = await page.$("#exito");
  expect(exitoEl).not.toBeNull();

  const texto = await page.$eval("#exito", (el) => el.textContent);
  expect(texto).toContain("éxito");

  // Captura automática de pantalla
  await saveScreenshot(page, "06_envio_exitoso");
}, 15000);
```

6.3 Capturas de Pantalla del Proceso (Puppeteer)

Las siguientes capturas fueron generadas automáticamente por Puppeteer durante la ejecución de los casos de prueba. Cada imagen documenta el estado del navegador en un momento específico del flujo de prueba.

0=U0 IC-E01 - Formulario de testimonios cargado correctamente en el navegador

0=U0 IC-E02 - Verificación de visibilidad de todos los campos del formulario

Testimoniales

Nombre**Correo Electrónico****Mensaje**

Capturas de Pantalla Validaciones y Errores

0=U0 TC-E03 - Mensajes de error al intentar enviar el formulario con campos vacíos

0=U0 TC-F04 - Error de validación al ingresar un correo electrónico con formato inválido

Testimoniales

Nombre

Juan Pérez

Correo Electrónico

correo-invalido

Ingresar un correo válido

Mensaje

Mi mensaje de prueba

Enviar Testimonial

Capturas de Pantalla - Llenado y Envío Exitoso

0=U0 TC-E05 - Formulario correctamente diligenciado con todos los campos válidos

0=U0 TC-F06 - Mensaje de confirmación tras el envío exitoso del testimonial

¡Testimonial enviado con éxito!

Gracias, **Pedro Sánchez**. Tu mensaje ha sido registrado.

[Enviar otro testimonial](#)

Testimoniales

Nombre

Tu nombre completo

Correo Electrónico

tu@correo.com

Mensaje

Escribe tu experiencia...

Enviar Testimonial

6.4 Resultados de Ejecución – Formulario Web

Comando de Ejecución

```
$ NODE_OPTIONS="--experimental-vm-modules" npx jest tests/formulario.test.js --no-coverage
```

Salida de Consola (Jest + Puppeteer Output)

```
console.log
  0=PE Servidor de prueba en: http://127.0.0.1:42345
  0=Uo Captura guardada: tests/screenshots/01_formulario_cargado.png
  0=Uo Captura guardada: tests/screenshots/02_campos_visibles.png
  0=Uo Captura guardada: tests/screenshots/03_errores_campos_vacios.png
  0=Uo Captura guardada: tests/screenshots/04_error_correo_invalido.png
  0=Uo Captura guardada: tests/screenshots/05_formulario_lleno.png
  0=Uo Captura guardada: tests/screenshots/06_envio_exitoso.png
  0=Uo Captura guardada: tests/screenshots/07_error_servidor_400.png
  0=Uo Captura guardada: tests/screenshots/08_formulario_movil.png

PASS tests/formulario.test.js (8.713 s)
  0=Yxp  Formulario Web - Testimoniales (Puppeteer)
    ' TC-F01: El formulario debe cargar correctamente (1086 ms)
    ' TC-F02: Todos los campos del formulario deben ser visibles (42 ms)
    ' TC-F03: Debe mostrar errores al enviar formulario vacio (1617 ms)
    ' TC-F04: Debe mostrar error con correo invalido (1652 ms)
    ' TC-F05: Debe permitir llenar todos los campos correctamente (772 ms)
    ' TC-F06: Debe enviar el formulario y mostrar mensaje de exito (1972 ms)
    ' TC-F07: El servidor debe rechazar datos vacios con status 400 (48 ms)
    ' TC-F08: El formulario debe verse correctamente en móvil (375px) (625 ms)

Test Suites: 1 passed, 1 total
Tests:      8 passed, 8 total
Time:       8.761 s
```

Tabla de Resultados Formulario

Caso	Descripción	Estado	Tiempo
TC-F01	El formulario debe cargar correctamente	PASS	1086 ms
TC-F02	Todos los campos del formulario deben ser visibles	PASS	42 ms
TC-F03	Debe mostrar errores al enviar formulario vacío	PASS	1617 ms
TC-F04	Debe mostrar error con correo inválido	PASS	1652 ms
TC-F05	Debe permitir llenar todos los campos correctamente	PASS	772 ms
TC-F06	Debe enviar el formulario y mostrar mensaje de éxito	PASS	1972 ms
TC-F07	El servidor debe rechazar datos vacíos con status 400	PASS	48 ms
TC-F08	El formulario debe verse correctamente en móvil (375px)	PASS	625 ms

8 / 8 Pruebas Pasadas – Tiempo Total: 8.761 s

7. Resumen Consolidado de Resultados

Script	Herramienta	Total	Pasadas	Fallidas	Tiempo
Script 1 – API REST	Jest + Supertest	15	15	0	0.307 s
Script 2 – Formulario Web	Jest + Puppeteer	8	8	0	8.761 s
TOTAL GENERAL	—	23	23	0	9.068 s

23

Casos de Prueba

100%

Tasa de Éxito

0

Pruebas Fallidas

8

Capturas de Pantalla

Análisis del Proyecto

Se analizó la estructura del proyecto: rutas Express, controladores, modelos Sequelize y vistas Pug. Se identificaron los endpoints y formularios a probar.

Paso 2

Diseño de Casos de Prueba

Se diseñaron 23 casos de prueba cubriendo escenarios positivos, negativos, de borde y de estructura, siguiendo el estándar TC-XX para API y TC-FXX para formularios.

Paso 3

Configuración del Entorno

Se instalaron Jest, Supertest, Puppeteer y PDFKit. Se configuró jest.config.js para soporte de módulos ES (ESM) con --experimental-vm-modules.

Paso 4

Implementación Script API

Se creó tests/app.test.js con una app Express aislada (mock de datos) y 15 casos de prueba usando Supertest para peticiones HTTP sin base de datos real.

Paso 5

Implementación Script Formulario

Se creó tests/formulario.test.js con un servidor Express de prueba, formulario HTML completo y 8 casos de prueba usando Puppeteer con capturas automáticas.

Paso 6

Ejecución y Verificación

Se ejecutaron ambos scripts: 15/15 API (0.307s) y 8/8 formulario (8.761s). Se generaron 8 capturas de pantalla automáticas.

Paso 7

Generación del Informe

Se generó este documento PDF con PDFKit, incluyendo código fuente, tablas de resultados, capturas de pantalla y análisis de resultados.

9. Reflexión sobre la Utilidad de las Pruebas Automatizadas

La implementación de este taller permitió evidenciar de manera práctica el valor real de las pruebas automatizadas en el ciclo de desarrollo de software. A continuación se presentan las reflexiones más relevantes derivadas de la experiencia.

%¶ Detección Temprana de Errores

Las pruebas automatizadas permiten detectar regresiones y errores en el momento en que se introducen, no cuando llegan a producción. En este proyecto, la validación del controlador de testimoniales (campos vacíos) fue verificada automáticamente en milisegundos.

%¶ Documentación Viva del Comportamiento

Los scripts de prueba actúan como documentación ejecutable del sistema. Cada caso de prueba describe con precisión qué debe hacer el sistema ante una entrada específica, siendo más confiable que documentación estática que puede quedar desactualizada.

%¶ Confianza para Refactorizar

Con una suite de 23 pruebas, cualquier cambio en los controladores o rutas puede verificarse instantáneamente. Esto reduce el miedo a refactorizar y permite mejorar el código con seguridad.

%¶ Separación de Responsabilidades en Testing

La combinación de Jest+Supertest (lógica de negocio/API) y Puppeteer (UI/UX) demuestra que diferentes capas del sistema requieren diferentes estrategias de prueba. No existe una herramienta única que cubra todos los escenarios.

%¶ Aislamiento como Buena Práctica

El uso de mocks y servidores de prueba aislados garantiza que las pruebas sean deterministas, rápidas y no dependan de infraestructura externa (base de datos, red). Esto es fundamental para integración continua (CI/CD).

%¶ ROI de la Automatización

Aunque la inversión inicial en escribir pruebas requiere tiempo, el retorno es exponencial: cada ejecución futura (que puede ser automática en cada commit) ahorra horas de pruebas manuales y reduce el costo de los errores en producción.

- 1. Se diseñaron e implementaron exitosamente 2 scripts de prueba automatizados: uno para API REST (15 casos) y uno para formulario web (8 casos), logrando una tasa de éxito del 100% (23/23 pruebas pasadas).
- 2. Jest y Supertest demostraron ser herramientas complementarias ideales para pruebas de integración de APIs Node.js, permitiendo verificar códigos HTTP, estructuras de respuesta y comportamientos de validación sin dependencia de base de datos.
- 3. Puppeteer permitió automatizar la interacción con el formulario web de manera que simula fielmente el comportamiento de un usuario real, incluyendo validaciones JavaScript del lado del cliente y capturas de pantalla automáticas.
- 4. La estrategia de aislamiento mediante mocks y servidores de prueba independientes garantizó pruebas deterministas, rápidas y reproducibles en cualquier entorno.
- 5. Las 8 capturas de pantalla generadas automáticamente por Puppeteer constituyen evidencia visual del comportamiento del sistema, complementando la documentación técnica del taller.
- 6. La automatización de pruebas es una inversión estratégica que mejora la calidad del software, reduce costos de mantenimiento y aumenta la confianza del equipo de desarrollo para realizar cambios y mejoras continuas.

Estudiante AA1

Automatización de Pruebas de Software

Fecha de entrega: 4 de mayo de 2026